

Estructura de datos

Listas doblemente enlazadas

Dr. Rodrigo Vázquez López

Universidad Autónoma de la Ciudad de México
Plantel San Lorenzo Tezonco

Semestre 2026-II

UACM

Universidad Autónoma
de la Ciudad de México

NADA HUMANO ME ES AJENO

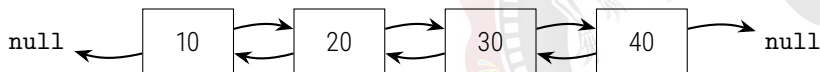
- 1 Listas doblemente enlazadas
- 2 Implementación
- 3 Recorrido
- 4 Agregar nodos
- 5 Obtener nodos
- 6 Eliminar nodos
- 7 Buscar nodos
- 8 Limpiar lista



Lista doblemente enlazada

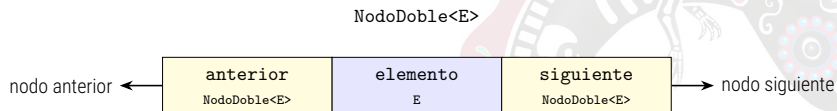
Definición

Una **lista doblemente enlazada** está formada por **nodos** que mantienen **referencias** al nodo **anterior** y al **siguiente**.



- Permite **realizar recorridos** en **ambas direcciones**.
- Cada nodo mantiene **dos referencias**.
- La lista conserva **referencias** a **inicio** y **fin**.

Estructura de un nodo doble



Objeto autorreferenciado

`NodoDoble<E>` contiene dos referencias a objetos de su mismo tipo.

Modelado de un nodo doble

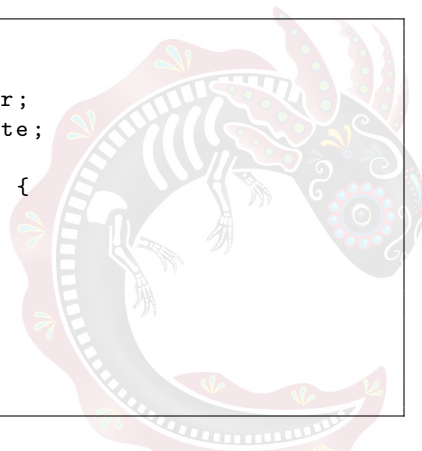
NodoDoble
-elemento : E -anterior : NodoDoble -siguiente : NodoDoble
+Nodo(elemento : E) +getElemento() : E +setElemento(elemento : E) +getAnterior() : NodoDoble +setAnterior(anterior : NodoDoble) +getSiguiente() : NodoDoble +setSiguiente(siguiente : NodoDoble)

E

anterior/siguiente

Clase NodoDoble<E>

```
public class NodoDoble<E> {  
    private E elemento;  
    private NodoDoble<E> anterior;  
    private NodoDoble<E> siguiente;  
  
    public NodoDoble(E elemento) {  
        this.elemento = elemento;  
        this.anterior = null;  
        this.siguiente = null;  
    }  
  
    // getters y setters  
}
```



¡Importante!

Un `Nodo<E>` de una lista simple mantiene **una referencia**, un `NodoDoble<E>` mantiene **dos**.

Conexión entre nodos

Definición

Para conectar **dos nodos** deben establecerse dos referencias:

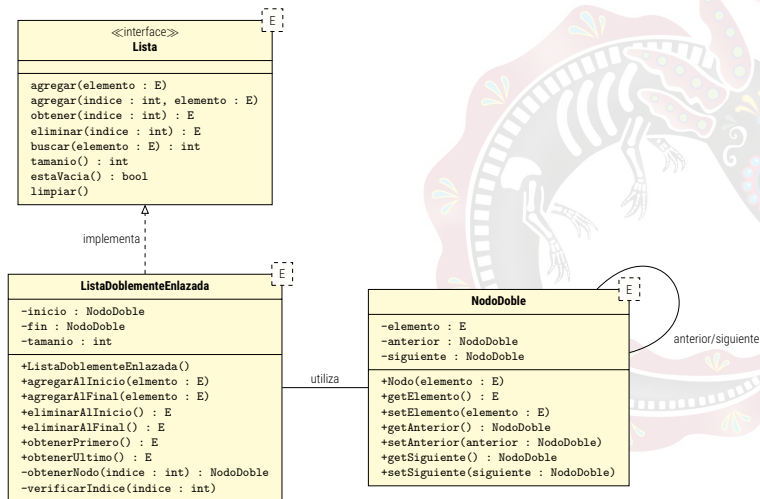
- 1: $\text{nodoA.siguiente} \leftarrow \text{nodoB}$
- 2: $\text{nodoB.anterior} \leftarrow \text{nodoA}$



Invariante

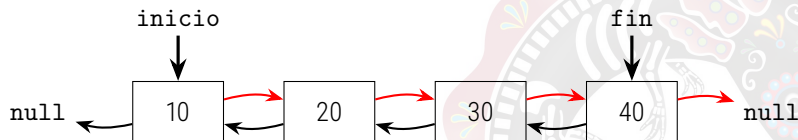
Si $\text{A.siguiente} == \text{B}$, entonces $\text{B.anterior} == \text{A}$

Diagrama de clases



Representación interna

`ListaDoblementeEnlazada<E>` mantiene referencias a **ambos extremos**.



Invariantes

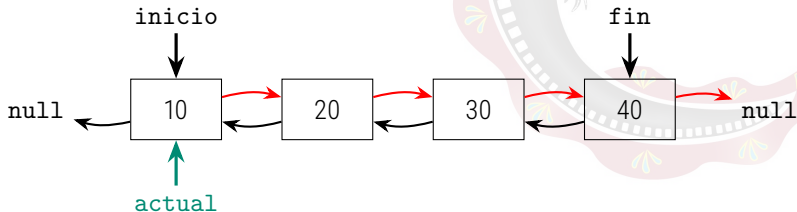
- `inicio.anterior == null`.
- `fin.siguiente == null`.
- Si **está vacía**: `inicio == fin == null`.
- `tamano` representa el **número de nodos**.

Recorrido desde el inicio

- 1: $actual \leftarrow inicio$
- 2: **while** $actual \neq null$ **do**
- 3: procesar $actual$
- 4: $actual \leftarrow actual.siguiete$
- 5: **end while**

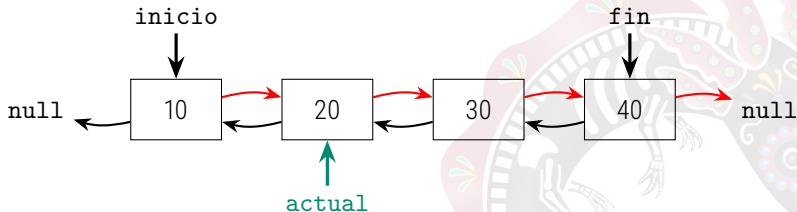
Ejemplo:

Asignamos la referencia `inicio` al nodo `actual`:

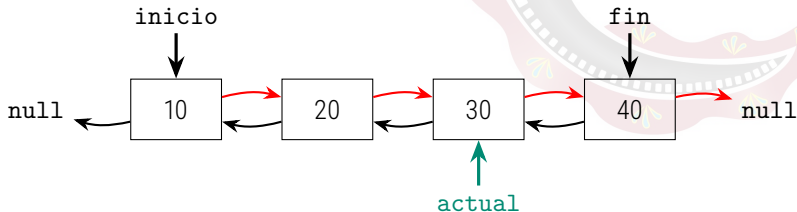


Recorrido desde el inicio

Mientras `actual != null` entonces `actual = actual.siguiente`:

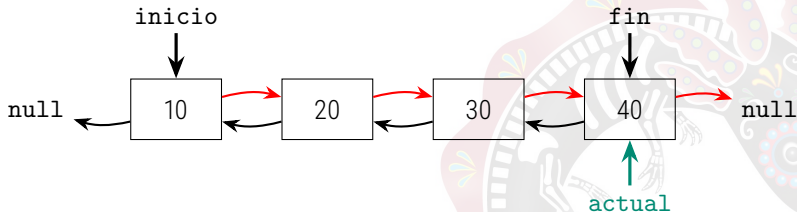


Mientras `actual != null` entonces `actual = actual.siguiente`:

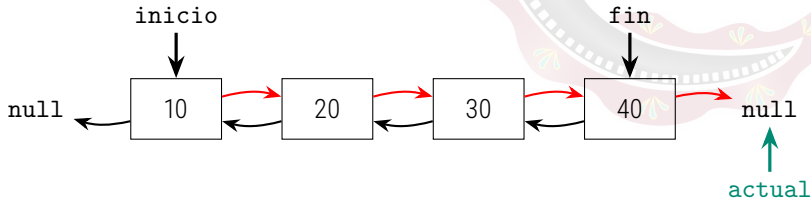


Recorrido desde el inicio

Mientras `actual != null` entonces `actual = actual.siguiente`:



Cuando `actual = null` termina el recorrido.

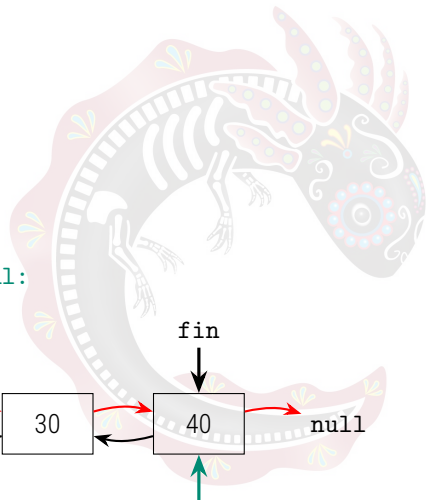
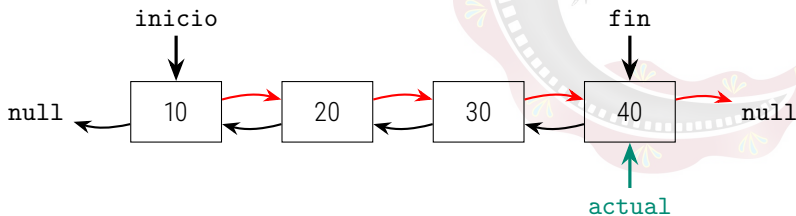


Recorrido desde el final

- 1: $actual \leftarrow fin$
- 2: **while** $actual \neq null$ **do**
- 3: procesar $actual$
- 4: $actual \leftarrow actual.anterior$
- 5: **end while**

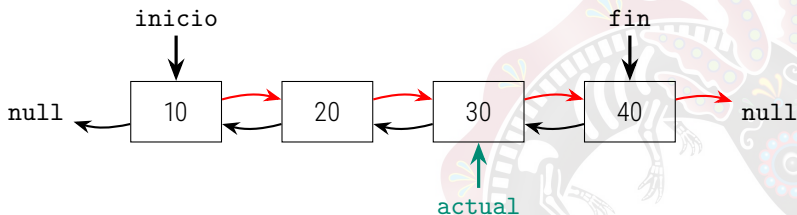
Ejemplo:

Asignamos la referencia **fin** al nodo **actual**:

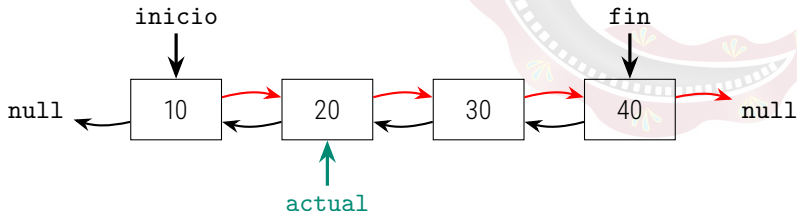


Recorrido desde el final

Mientras `actual != null` entonces `actual = actual.anterior`:

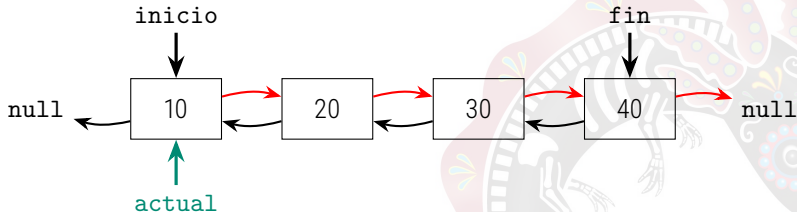


Mientras `actual != null` entonces `actual = actual.anterior`:

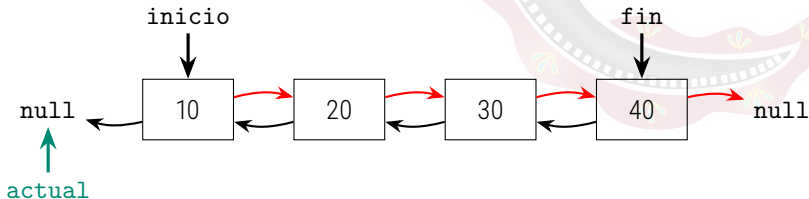


Recorrido desde el final

Mientras `actual != null` entonces `actual = actual.anterior`:



Cuando `actual = null` termina el recorrido.



Método obtenerNodo(indice)

Require: indice

```

1: if indice < tamano/2 then
2:   actual ← inicio
3:   for i ← 0 hasta indice - 1 do
4:     actual ← actual.siguiente
5:   end for
6: else
7:   actual ← fin
8:   for i ← tamano - 1 hasta indice + 1 do
9:     actual ← actual.anterior
10:  end for
11: end if
12: return actual

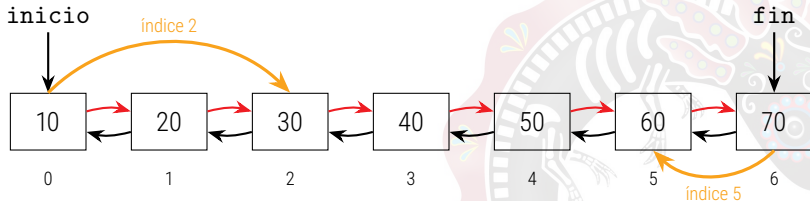
```



¡Importante!

Comenzar el **recorrido** desde el extremo **más cercano al índice solicitado**.

¿Desde dónde comenzar?



`obtenerNodo(2)` $\Rightarrow$ recorrer desde **inicio**

`obtenerNodo(5)` $\Rightarrow$ recorrer desde **fin**

Método agregar(elemento)

Require: elemento

- 1: $nuevo \leftarrow \text{NodoDoble}(\text{elemento})$
- 2: **if** la lista está vacía **then**
- 3: $inicio \leftarrow nuevo$
- 4: $fin \leftarrow nuevo$
- 5: **else**
- 6: $nuevo.anterior \leftarrow fin$
- 7: $fin.siguiente \leftarrow nuevo$
- 8: $fin \leftarrow nuevo$
- 9: **end if**
- 10: $tamano \leftarrow tamano + 1$

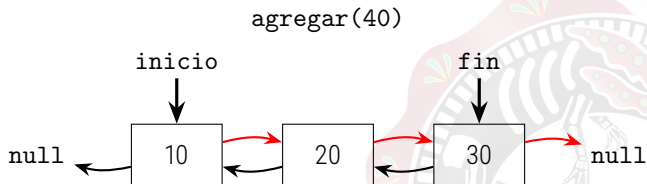


¡Importante!

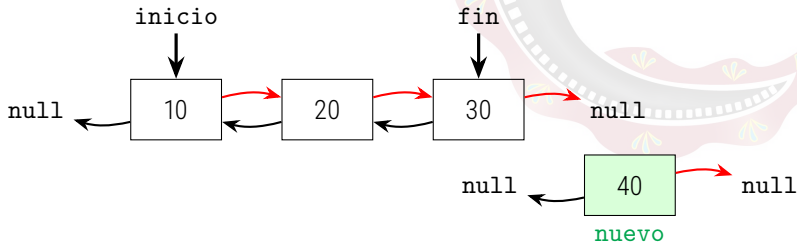
No es necesario recorrer la lista para **localizar** el **último nodo**.

Agregar al final

Ejemplo:

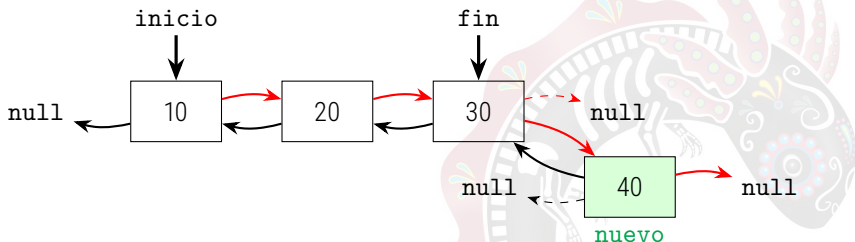


Agregamos un nodo **nuevo**:

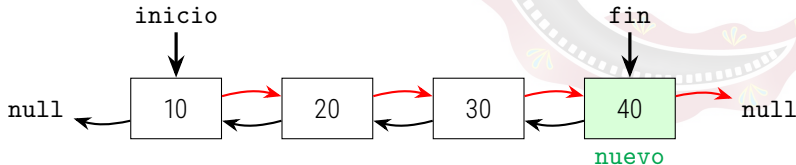


Agregar al final

Asignamos `nuevo.anterior = fin` y `fin.siguiente = nuevo`:



Actualizamos la referencia `fin = nuevo`:



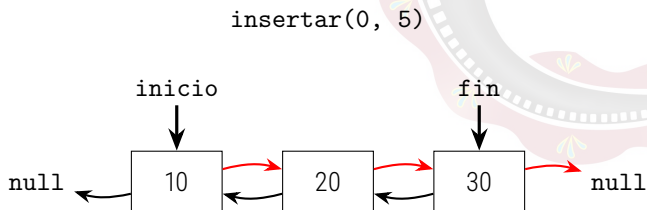
tamaño=4

Inserción al inicio

Para insertar en la posición 0:

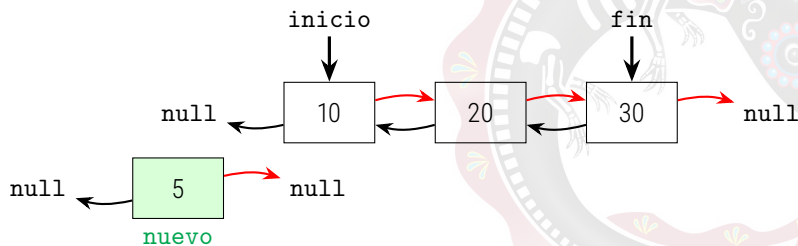
- 1: $nuevo \leftarrow \text{NodoDoble}(\text{elemento})$
- 2: $nuevo.siguiente \leftarrow inicio$
- 3: $inicio.anterior \leftarrow nuevo$
- 4: $inicio \leftarrow nuevo$
- 5: $tamaño \leftarrow tamaño + 1$

Ejemplo:



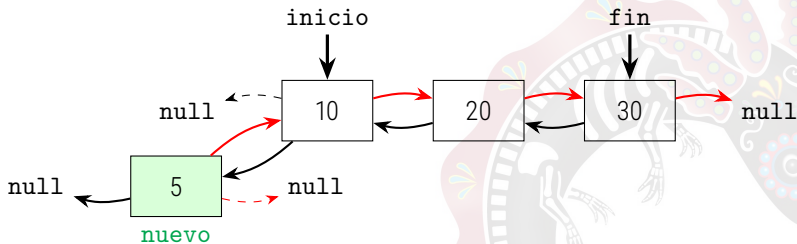
Inserción al inicio

Agregamos un nodo **nuevo**:

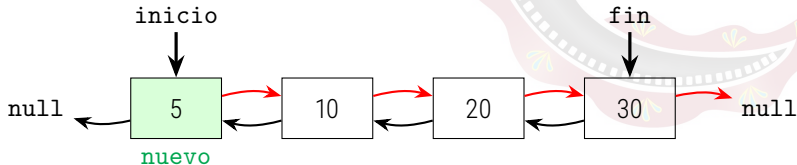


Inserción al inicio

Asignamos `nuevo.siguiente = inicio` e `inicio.anterior = nuevo`



Actualizamos la referencia `inicio = nuevo`:



tamaño=4

Inserción intermedia

Insertar un nuevo nodo antes de **actual**:

- 1: $actual \leftarrow obtenerNodo(indice)$
- 2: $anterior \leftarrow actual.anterior$
- 3: $nuevo \leftarrow NodoDoble(elemento)$
- 4: $nuevo.anterior \leftarrow anterior$
- 5: $nuevo.siguiente \leftarrow actual$
- 6: $anterior.siguiente \leftarrow nuevo$
- 7: $actual.anterior \leftarrow nuevo$
- 8: $tamano \leftarrow tamano + 1$

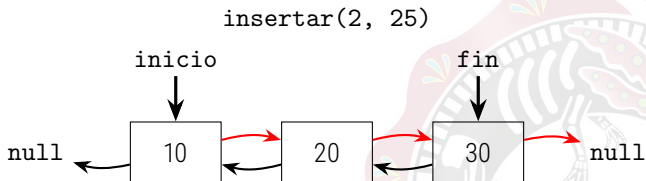


¡Importante!

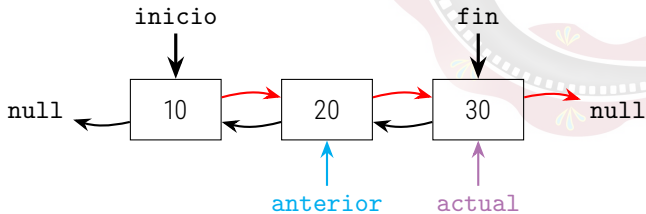
Una **inserción intermedia** requiere **actualizar** **cuatro referencias**.

Inserción de un nodo intermedio

Ejemplo:

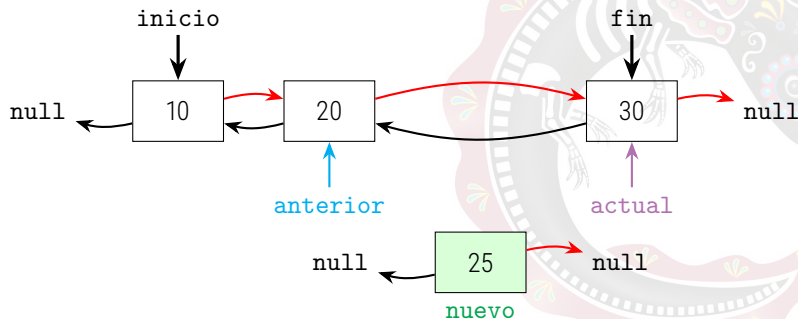


Apuntamos con **actual** a la posición donde se va a insertar y a la posición **anterior**



Inserción de un nodo intermedio

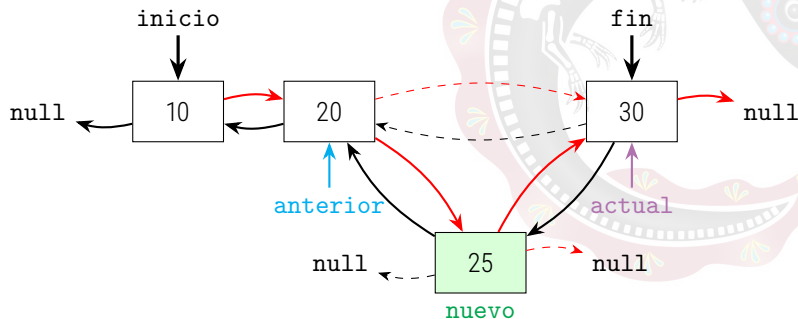
Creamos un nodo **nuevo**:



Inserción de un nodo intermedio

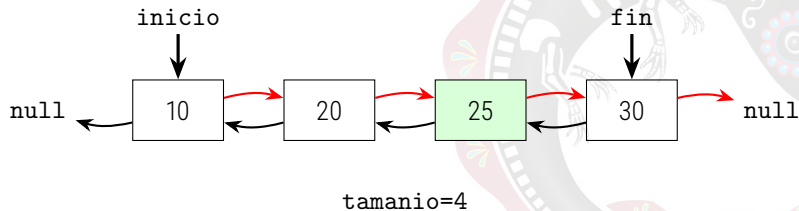
Actualizamos todas las referencias:

```
nuevo.anterior = anterior,  
nuevo.siguiente = actual,  
anterior.siguiente = nuevo y  
actual.anterior = nuevo:
```



Inserción de un nodo intermedio

Finalmente:



Método obtener(indice)

Require: indice

- 1: verificarIndice(indice)
- 2: $nodo \leftarrow obtenerNodo(indice)$
- 3: **return** $nodo.elemento$

Diferencia respecto a ListaArreglos

No existe acceso directo al elemento:

`elementos[indice]`

Por tanto, es necesario **recorrer** los enlaces.

¡Importante!

La **complejidad** es $O(n)$ aunque el recorrido puede comenzar desde el **extremo más cercano**.



Eliminación de un nodo

Al **eliminar un nodo** debemos considerar su **posición**:

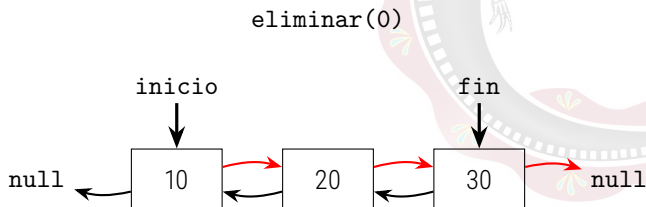
- 1 La lista contiene **un único nodo**.
- 2 Se elimina el **primer nodo**.
- 3 Se elimina el **último nodo**.
- 4 Se elimina un **nodo intermedio**.



Eliminar el primer nodo

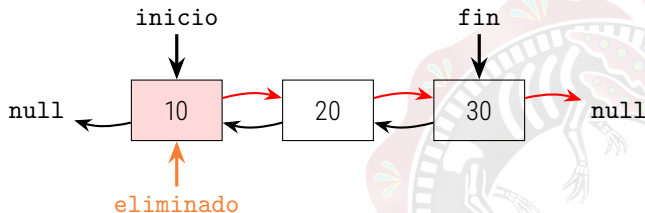
- 1: $\text{eliminado} \leftarrow \text{inicio}$
- 2: $\text{inicio} \leftarrow \text{inicio.siguiente}$
- 3: $\text{inicio.anterior} \leftarrow \text{null}$
- 4: $\text{tamanio} \leftarrow \text{tamanio} - 1$

Ejemplo:

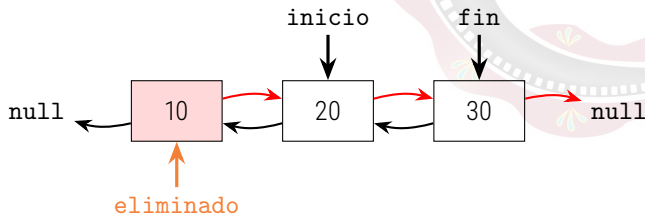


Eliminar el primer nodo

Marcamos el nodo `inicio` como `eliminado`:

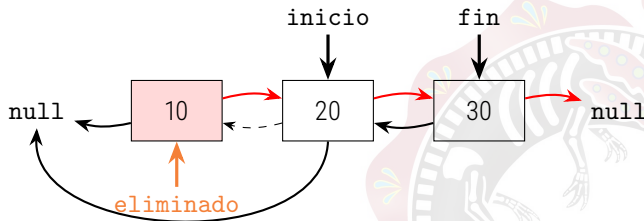


Cambiamos la referencia `inicio` = `eliminado.siguiente`:

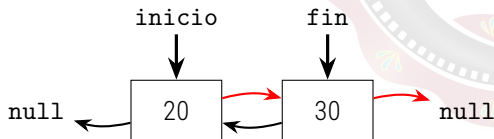


Eliminar el primer nodo

Cambiamos la referencia de `inicio.anterior = null`:



Finalmente:

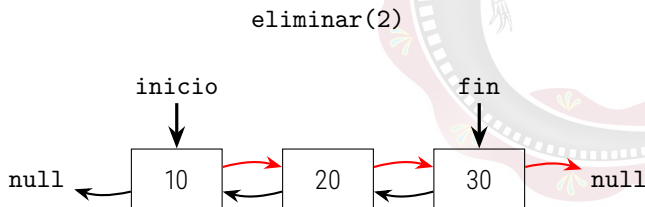


`tamano=2`

Eliminar el último nodo

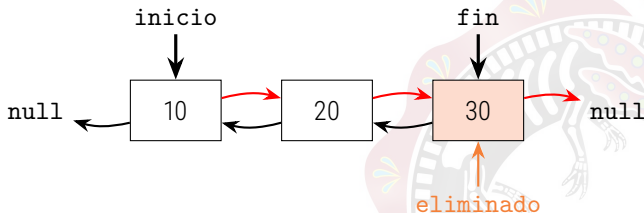
- 1: $eliminado \leftarrow fin$
- 2: $fin \leftarrow fin.anterior$
- 3: $fin.siguiete \leftarrow null$
- 4: $tamaño \leftarrow tamaño - 1$

Ejemplo:

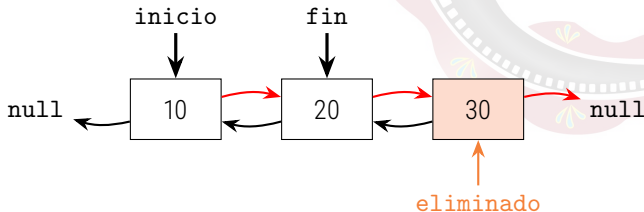


Eliminar el último nodo

Asignamos el nodo `fin` como **eliminado**:

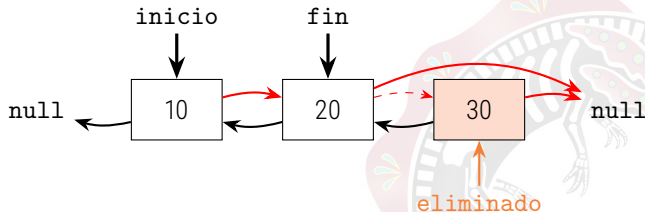


Asignamos `fin = fin.anterior`:

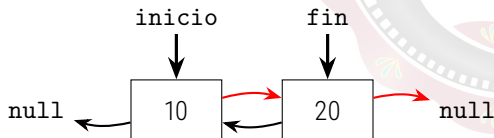


Eliminar el último nodo

Asignamos `fin.siguiente = null`:



Finalmente:

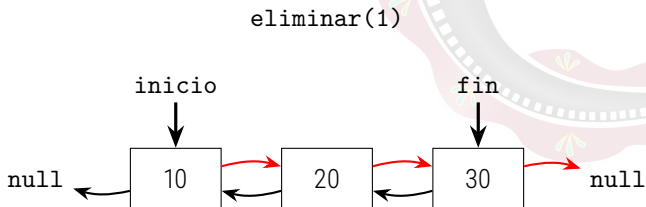


`tamanio=2`

Eliminación intermedia

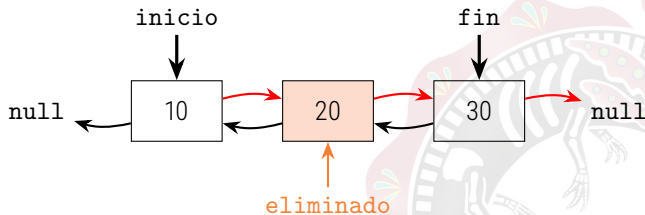
- 1: $\text{eliminado} \leftarrow \text{obtenerNodo}(\text{indice})$
- 2: $\text{anterior} \leftarrow \text{eliminado.anterior}$
- 3: $\text{siguiente} \leftarrow \text{eliminado.siguiete}$
- 4: $\text{anterior.siguiete} \leftarrow \text{siguiente}$
- 5: $\text{siguiente.anterior} \leftarrow \text{anterior}$
- 6: $\text{tamanio} \leftarrow \text{tamanio} - 1$

Ejemplo:

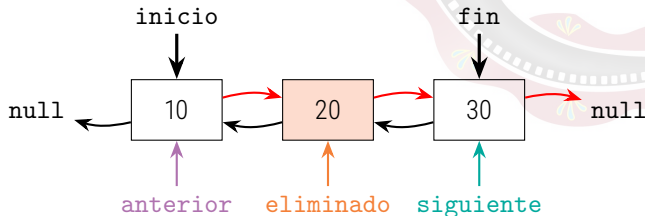


Eliminación intermedia

Marcamos el nodo como **eliminado**:

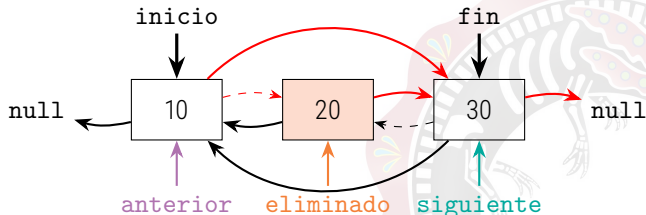


Marcamos `eliminado.anterior` como **anterior** y `eliminado.siguiente` como **siguiente**:

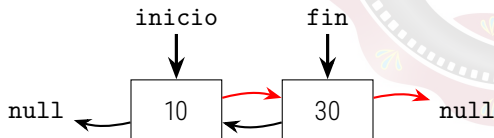


Eliminación intermedia

Asignamos `anterior.siguiente = siguiente` y `siguiente.anterior = anterior`:



Finalmente:



tamaño=2

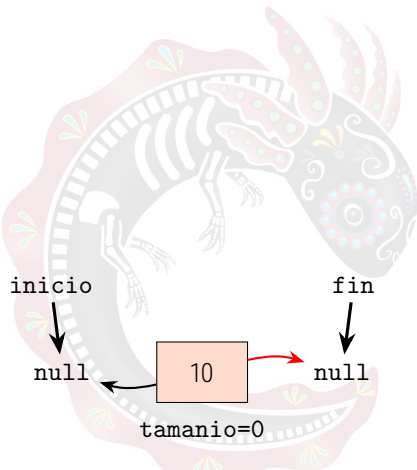
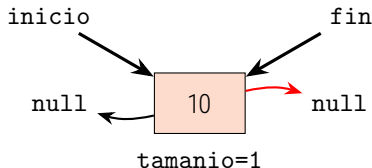
Eliminar el único elemento

```

1: if tamano = 1 then
2:   inicio  $\leftarrow$  null
3:   fin  $\leftarrow$  null
4:   tamano  $\leftarrow$  0
5: end if

```

Ejemplo:



¡Importante!

Al desaparecer el único nodo, deben actualizarse **ambos extremos**.

Método buscar(elemento)

Require: elemento

```

1: actual  $\leftarrow$  inicio
2: indice  $\leftarrow$  0
3: while actual  $\neq$  null do
4:   if actual.elemento = elemento then
5:     return indice
6:   end if
7:   actual  $\leftarrow$  actual.siguiente
8:   indice  $\leftarrow$  indice + 1
9: end while
10: return -1
    
```

$\boxed{10}_0 \rightarrow \boxed{20}_1 \rightarrow \boxed{30}_2$

buscar(30) $\rightarrow$ $\boxed{2}$

$O(n)$



Métodos tamaño() y estaVacia()

tamaño()

1: **return** *tamaño*

null $\leftarrow$ [10] $\leftrightarrow$ [20] $\leftrightarrow$ [30] $\rightarrow$ *null*

tamaño() = 3

$O(1)$

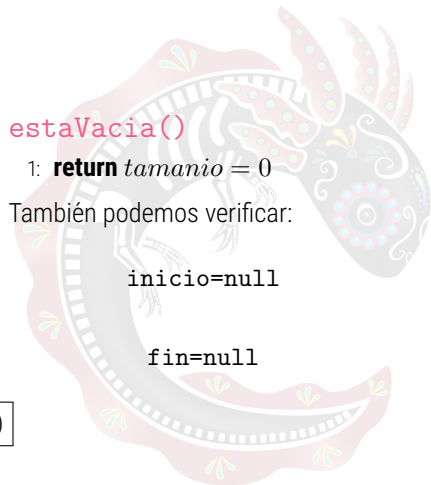
estaVacia()

1: **return** *tamaño* = 0

También podemos verificar:

inicio=*null*

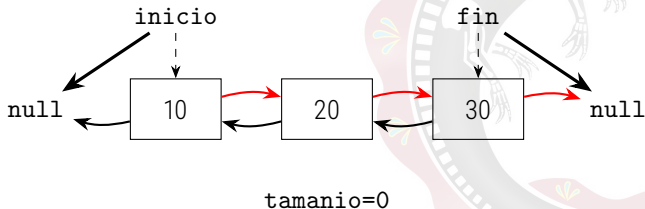
fin=*null*



Método limpiar()

Para **eliminar** lógicamente **todos los elementos**:

- 1: $inicio \leftarrow null$
- 2: $fin \leftarrow null$
- 3: $tamaño \leftarrow 0$



Java

Los nodos **dejan de ser accesibles** desde la lista y **pueden ser recuperados** posteriormente por el **recolector de basura**.

Actualización de referencias

Operación	Referencias principales
Agregar al final	3
Insertar al inicio	3
Insertar entre dos nodos	4
Eliminar al inicio	2
Eliminar al final	2
Eliminar nodo intermedio	2

¡Importante!

Después de **modificar** la estructura **debe mantenerse**:

$$A.\text{siguiente} = B \iff B.\text{anterior} = A$$

Lista simple vs. lista doble

Característica	Lista simple	Lista doble
Referencias por nodo	1	2
Recorrido hacia adelante	Sí	Sí
Recorrido hacia atrás	No	Sí
Referencia a fin	No*	Sí
Agregar al final	$O(n)$	$O(1)$
Memoria por nodo	Menor	Mayor

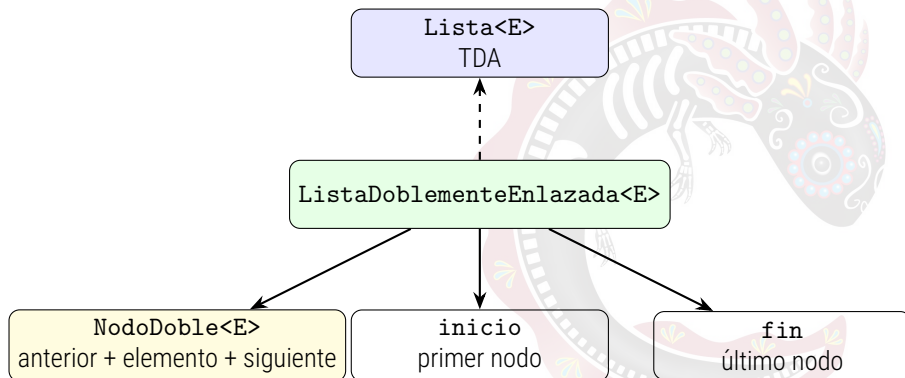
* Para las implementaciones desarrolladas en el curso.

Comparación de operaciones

Operación	ListaArreglos	ListaEnlazada	ListaDoblementeEnlazada
Obtener por índice	$O(1)$	$O(n)$	$O(n)$
Buscar	$O(n)$	$O(n)$	$O(n)$
Insertar al inicio	$O(n)$	$O(1)$	$O(1)$
Agregar al final	$O(1)^*$	$O(n)$	$O(1)$
Eliminar al inicio	$O(n)$	$O(1)$	$O(1)$
Tamaño	$O(1)$	$O(1)$	$O(1)$

* Excepto cuando es necesario redimensionar el arreglo.

Implementación de una lista doblemente enlazada



Idea fundamental

La lista doblemente enlazada utiliza **más referencias** que una lista simple, pero permite **recorrer y modificar** la estructura desde ambos extremos.

Fin de la presentación

¿Preguntas o comentarios?

